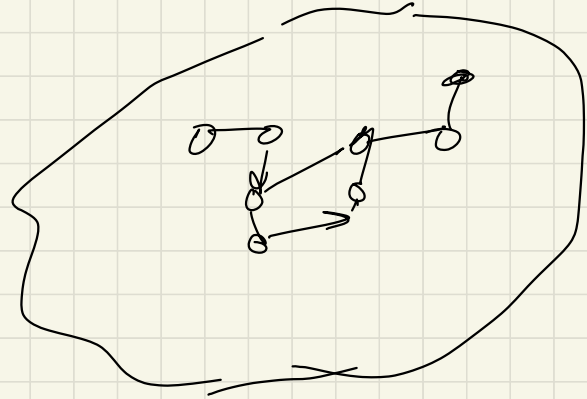
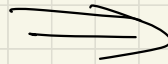
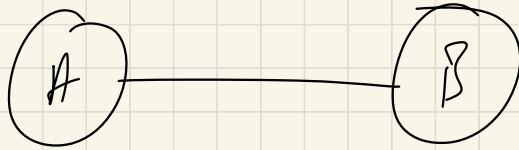
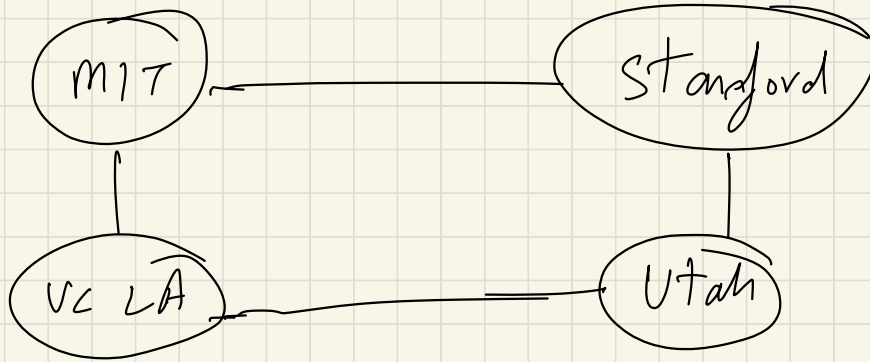


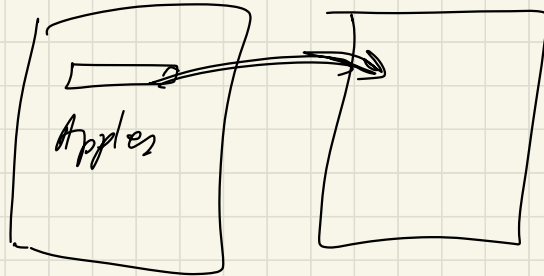

Computer Networking Course

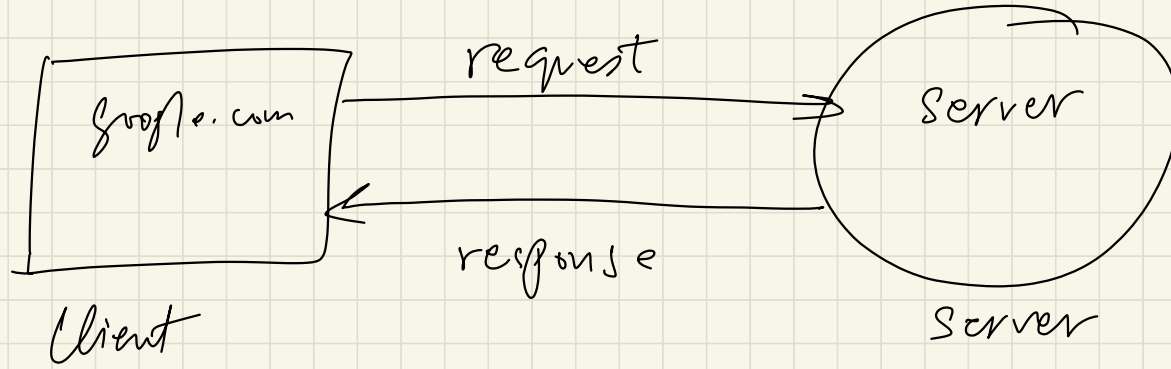


How did it start?



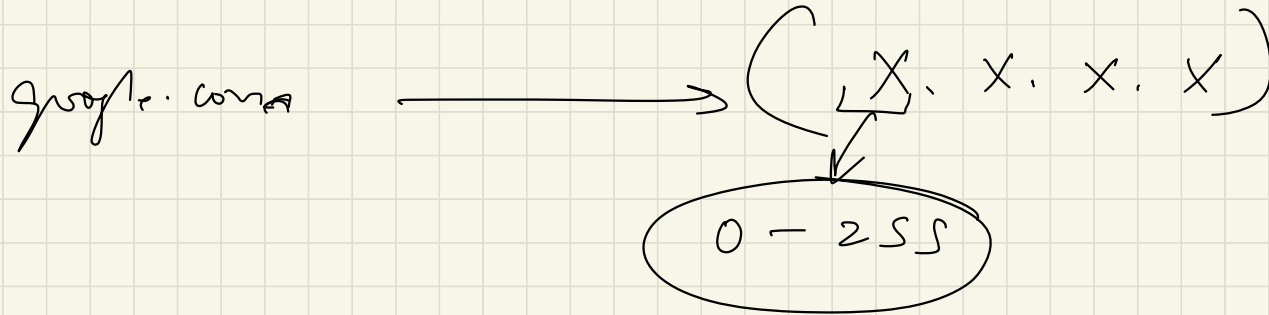
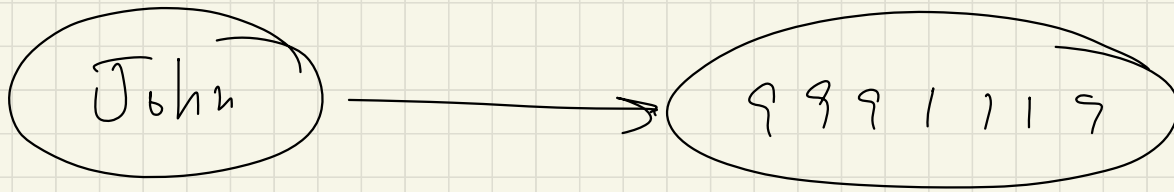
TCP/IP



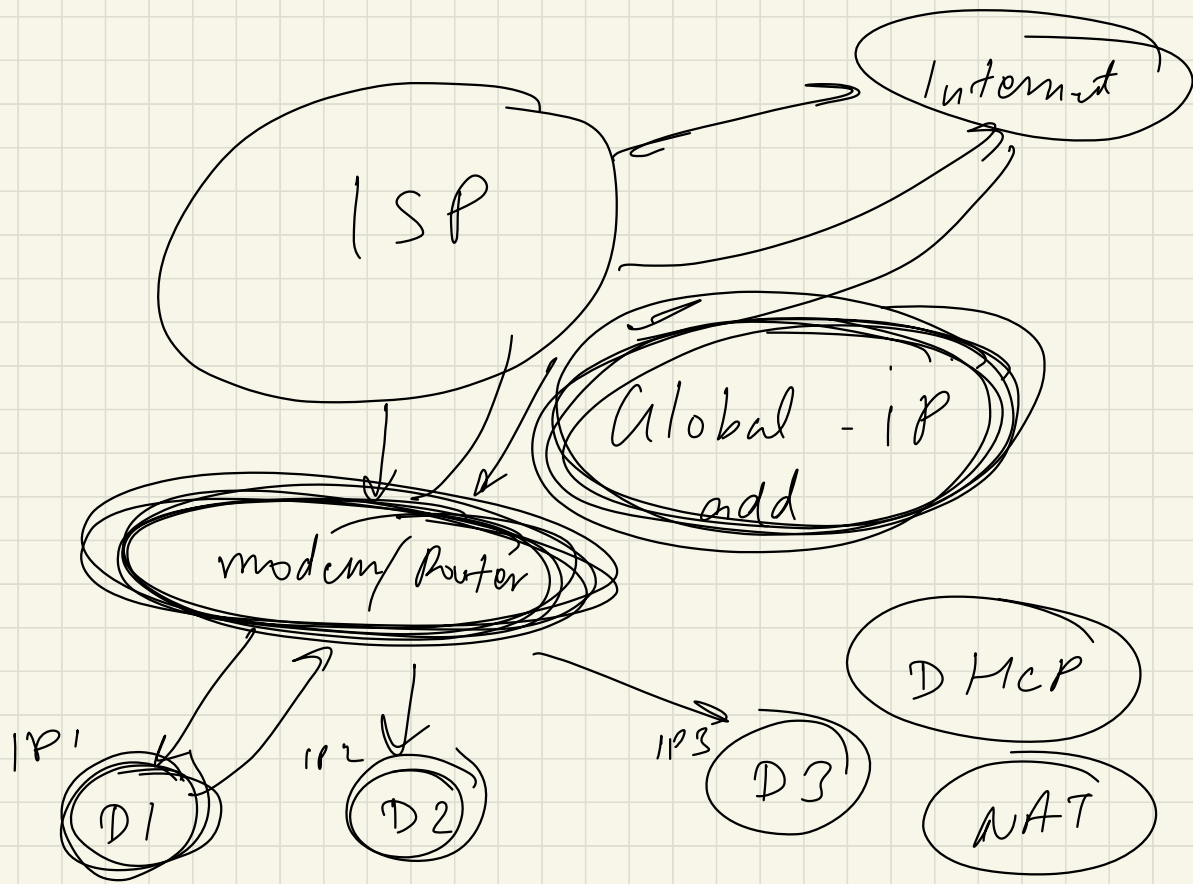


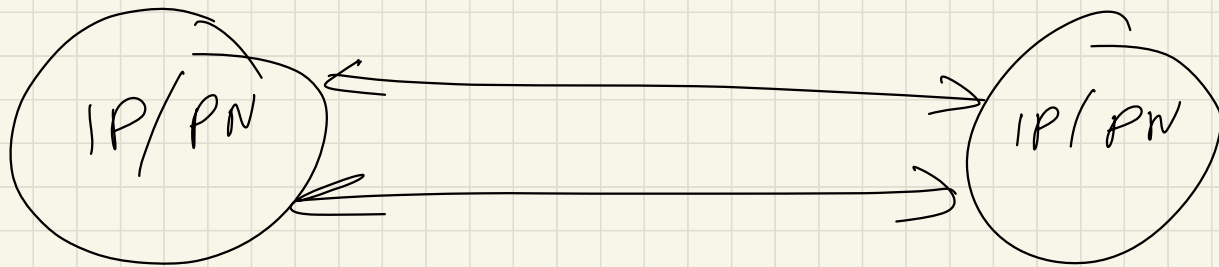
TCP, UDP, HTTP

Packets,



\$ cur.l if config.me - s





$$\text{Total} = 2^{16} \approx 65,000$$

$$\text{HTTP} = 80$$

$$\text{MongoDB} = 27017$$

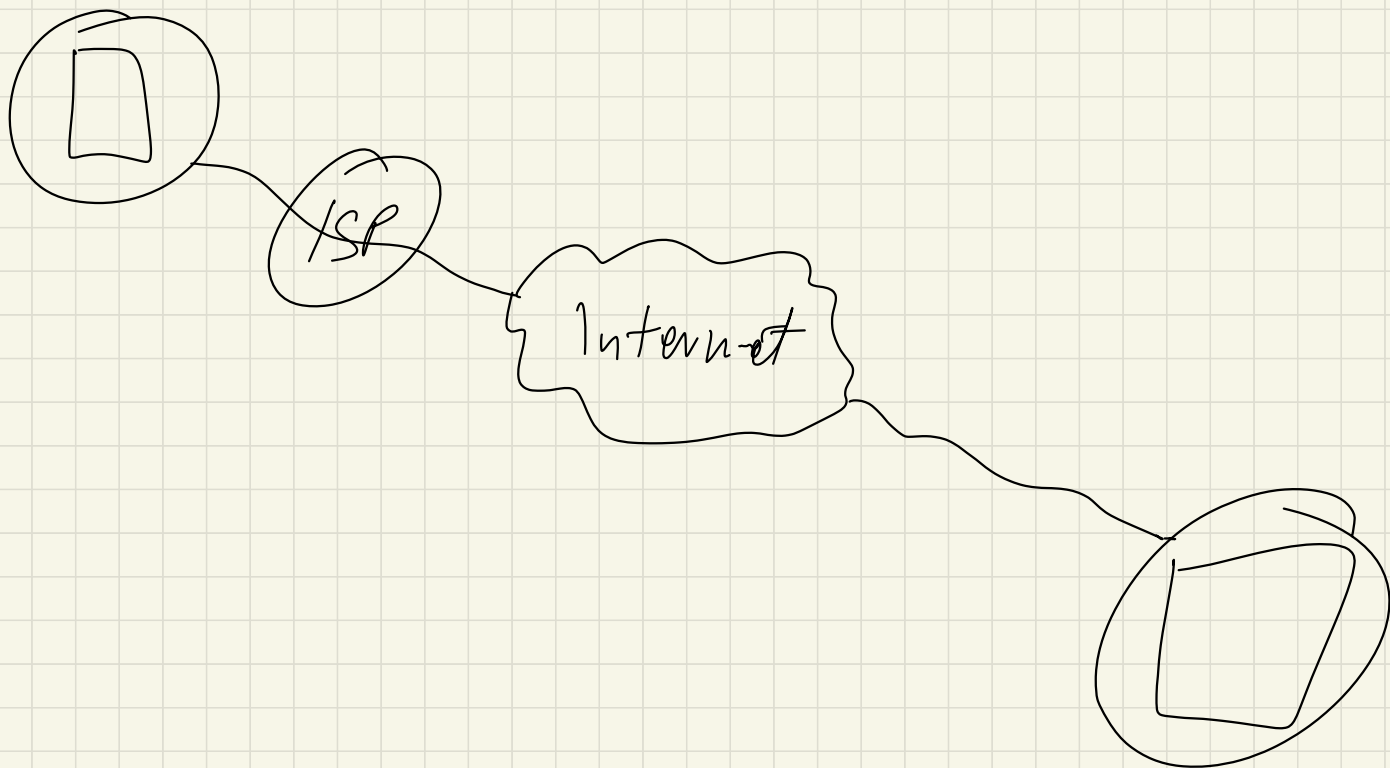
$$\text{SQL} = 1433$$

0 — 1023

reserved ports.

1024 — 49152
applications

remaining $\rightarrow$ you can use.



$$1 \text{ Mbps} = 1\,000\,000 \text{ bits/s}$$

$$1 \text{ Gbps} = 10^9 \text{ bits/s}$$

$$1 \text{ kbps} = 1000 \text{ bits/s}$$

Physically:

Optical fibre cables, Coaxial cables

Wireless:

Bluetooth, Wifi, 3G, 4G LTE, 5G

LAN : Small house / office
Ethernet , wifi

MAN : Across a city

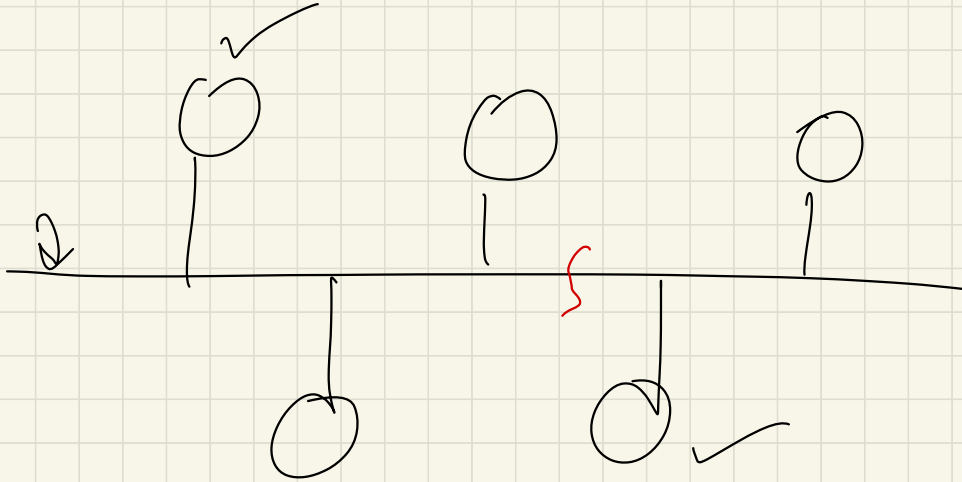
WAN : Across countries
optical fibre cables.

① SONET

② frame relay

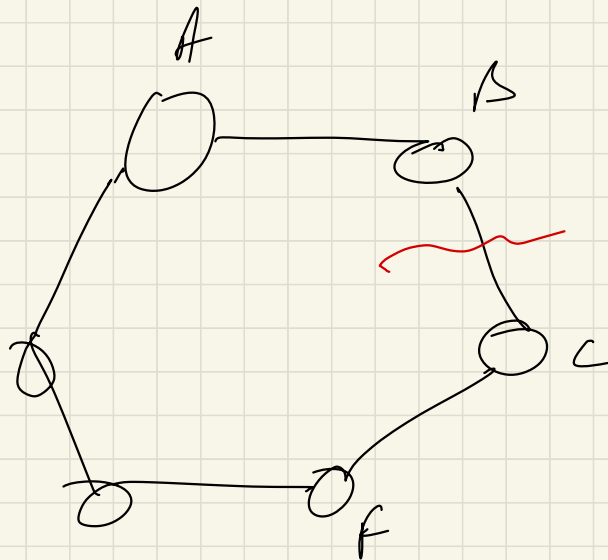
Topologies:

① BUS

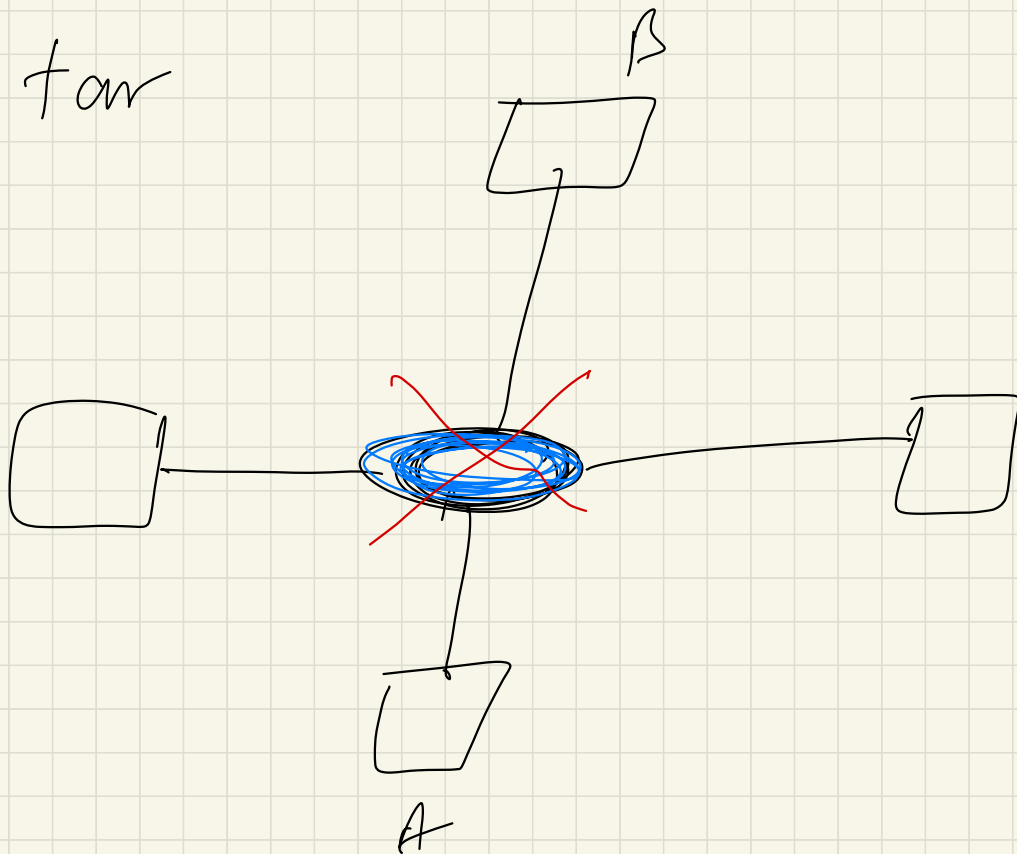


(2)

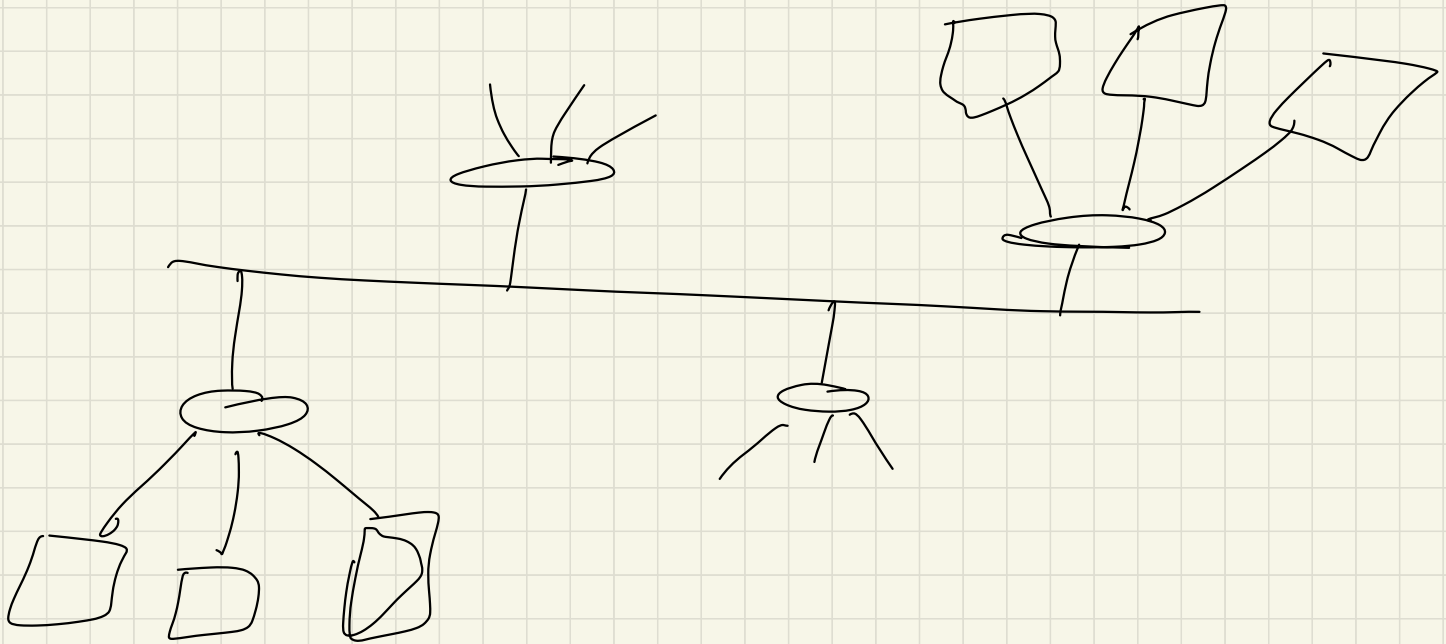
Ring



③ star

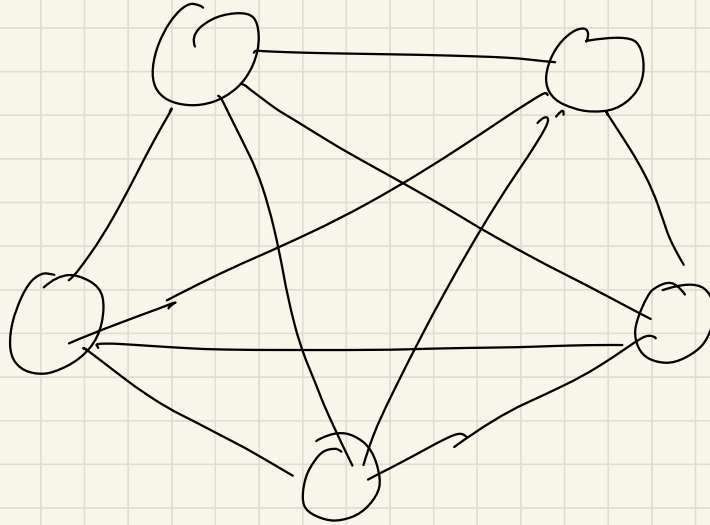


(4) Tree (Bus - star)



5

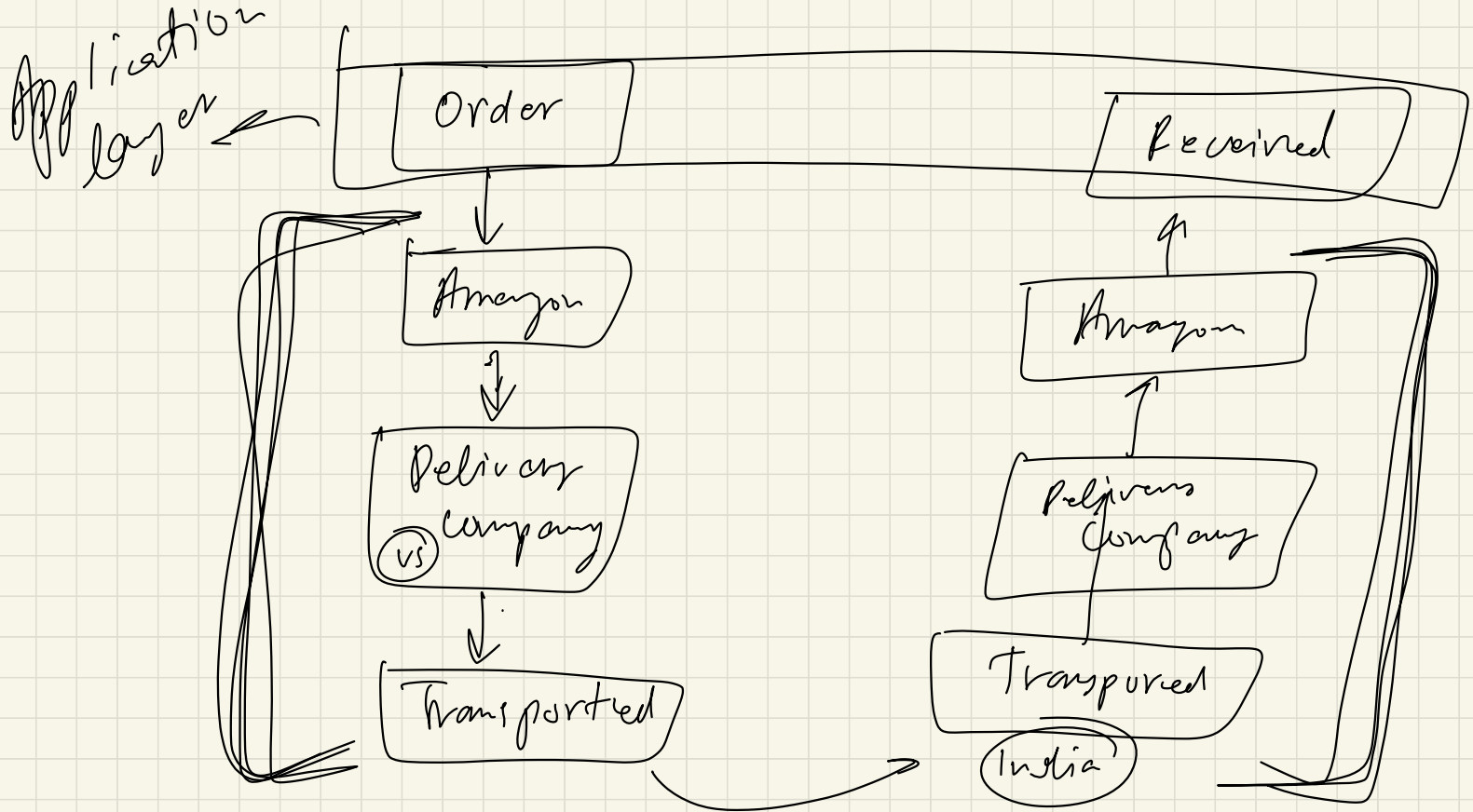
Mesh



* Expensive

* Scalability
issues

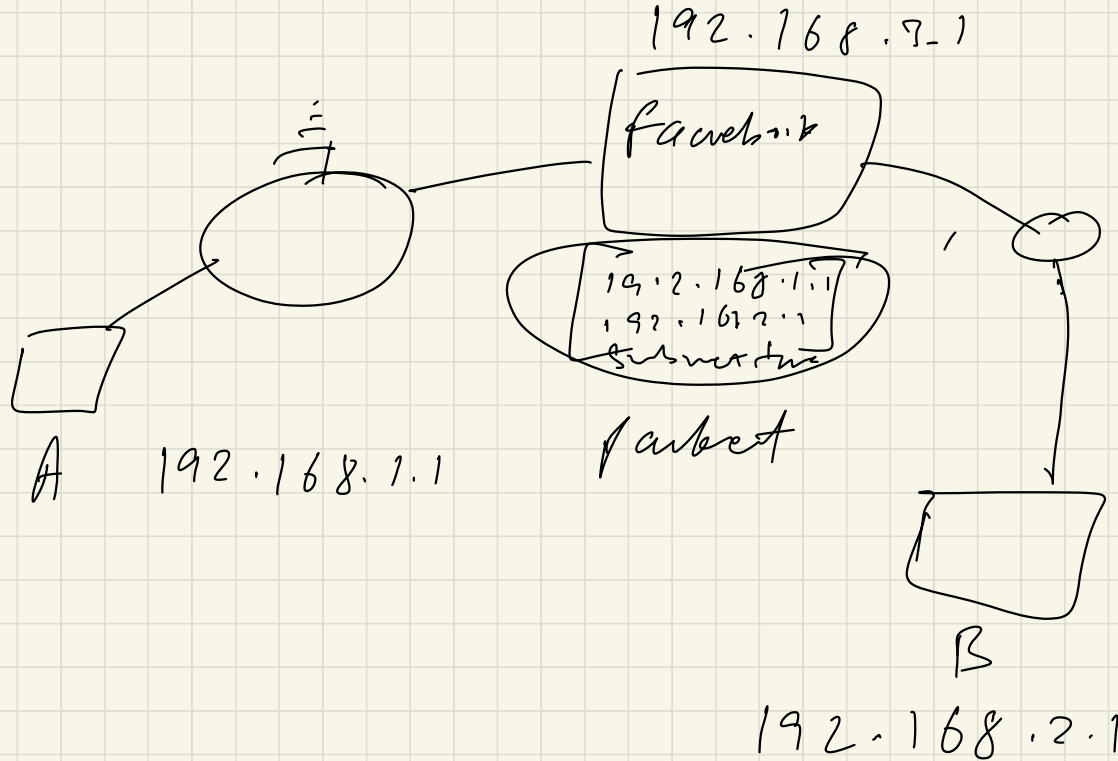
Structure of the Network



OSI Model

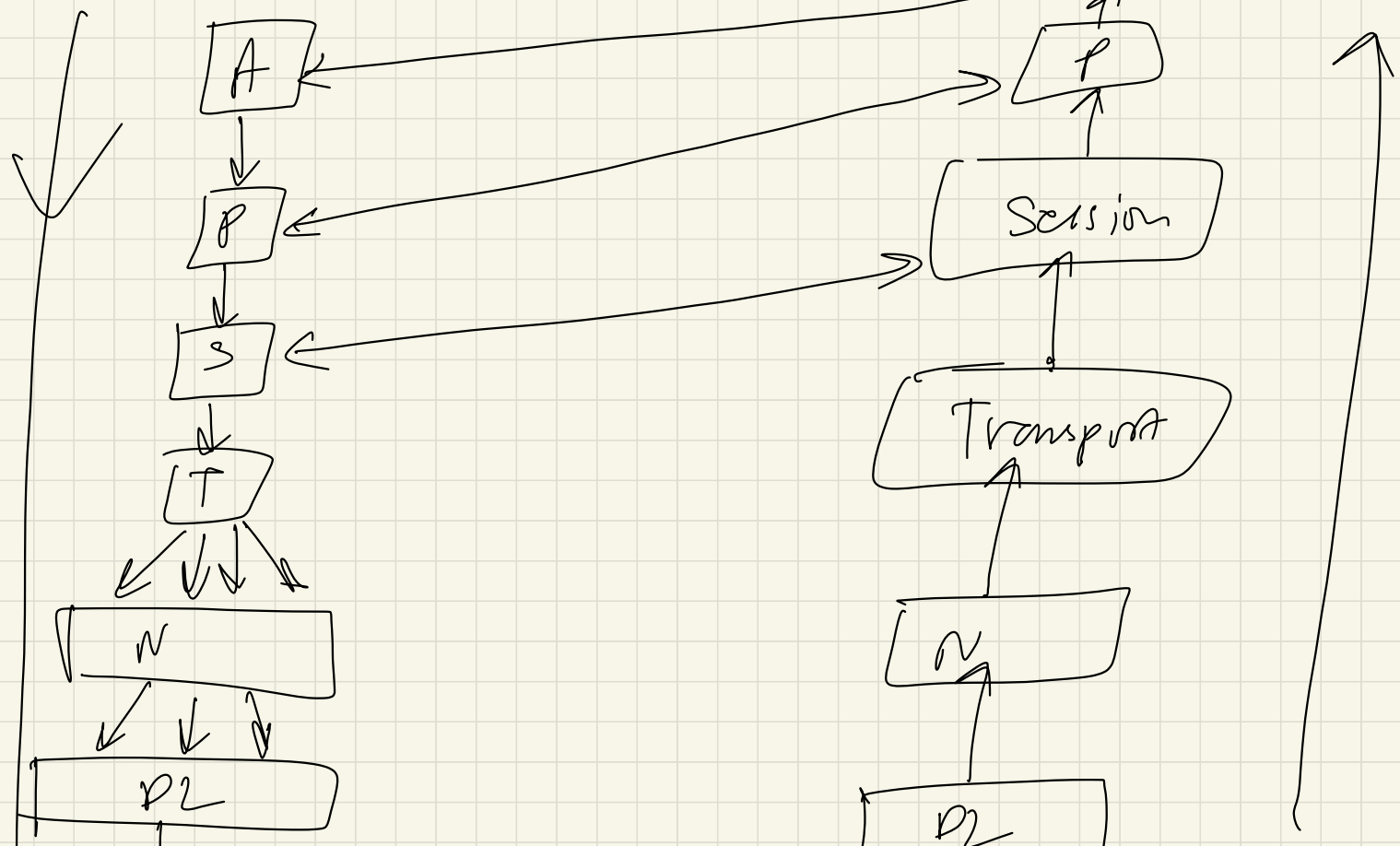
// make your own
notes from videw.

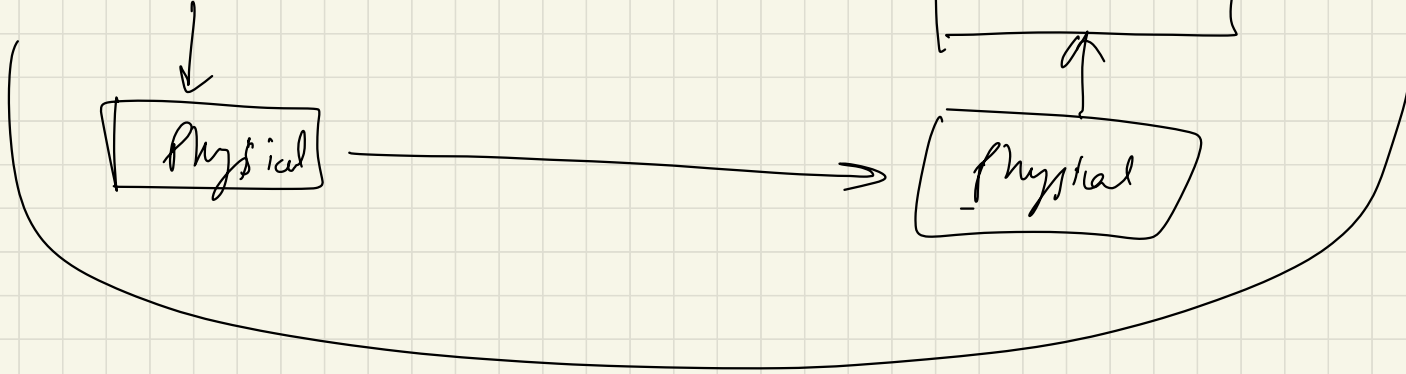
Application
↓
Presentation
↓
Session
↓
Transport
↓
Network
↓
Data Link
↓
Physical



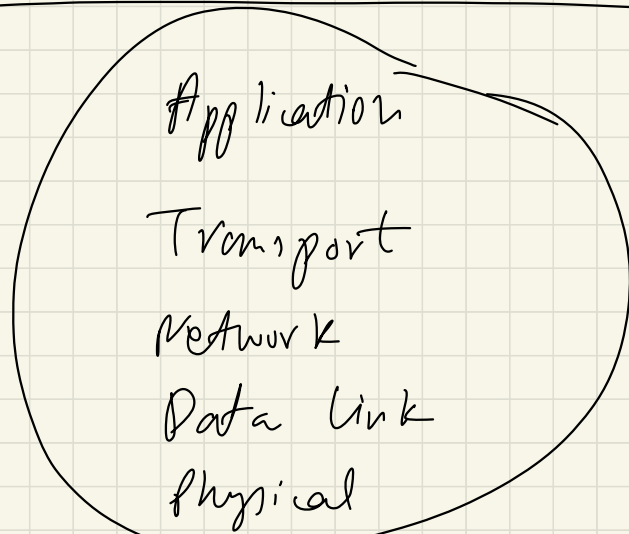
you

friend





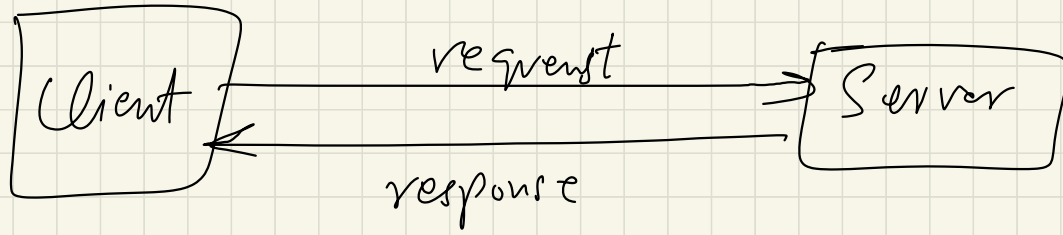
Another model : TCP/IP model



① Application layer :

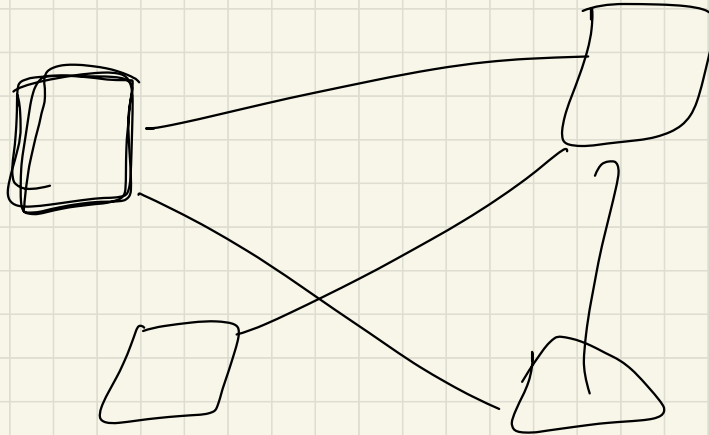
- ★ Users interact
- ★ Whatsapp, Browsers, etc.
- ★ where : Devices.
- ★ Protocols
- ★ Client - Server Architecture

①



②

P2P



Protocols:

Web protocols:

TCP/IP:

★ HTTP

★ DHCP

★ FTP

★ SMTP

★ POP3 & IMAPC

★ SSH

★ VNC

★ Telnet : Port 23

* UDP : stateless connection

Program : WhatsApp

Process :

Send a message

Record a video



Thread :

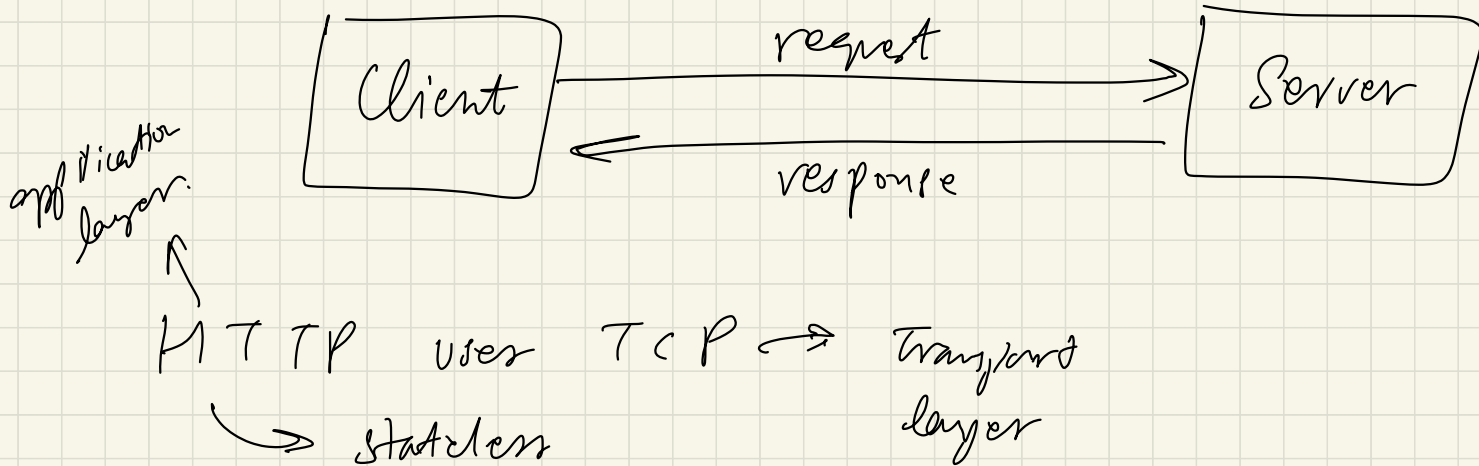
Set up the
page

Open cam

Sockets

ports: Ephemeral ports.

HTTP



HTTP methods:

- ① GET
- ② POST
- ③ PUT
- ④ DELETE

Status Codes:

- | | | |
|-----|---|---------------|
| 1xx | → | Informational |
| 2xx | → | Success |
| 3xx | → | Redirecting |
| 4xx | → | Client error |
| 5xx | → | Server error |

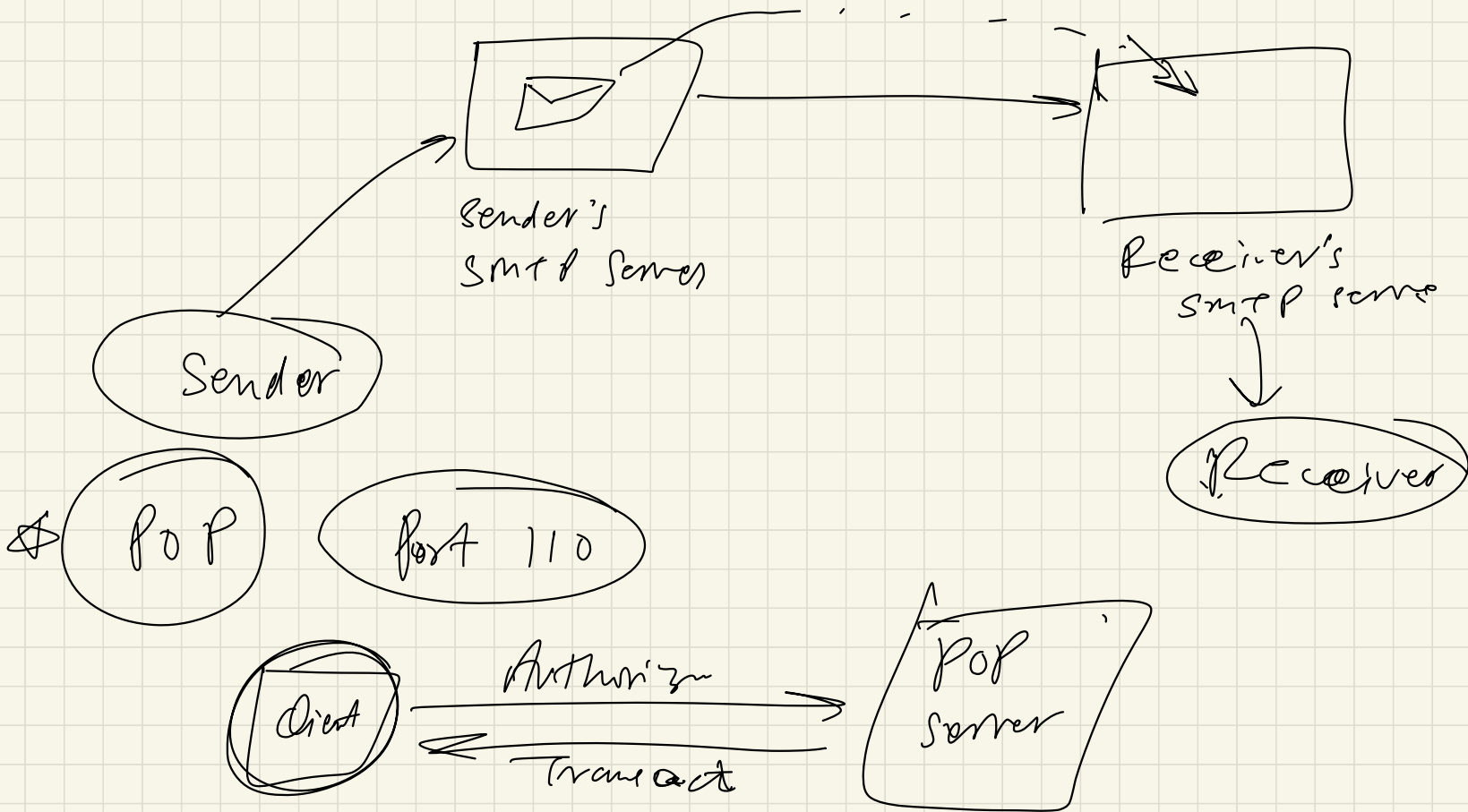
Cookies : Unique String.

Stored in my browser.

★ Third party cookies

How email works?

SMTP, POP3



★ I MAP

DNS

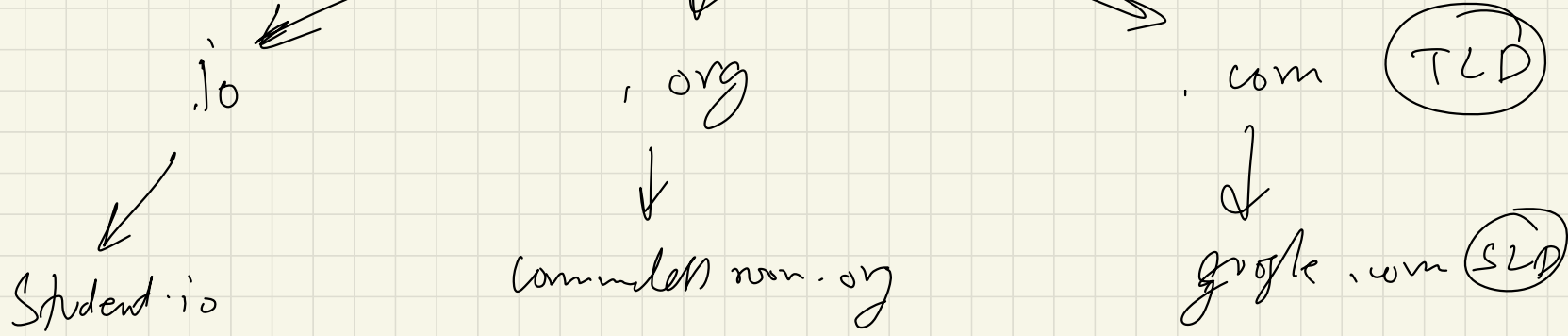
: Domain Name System

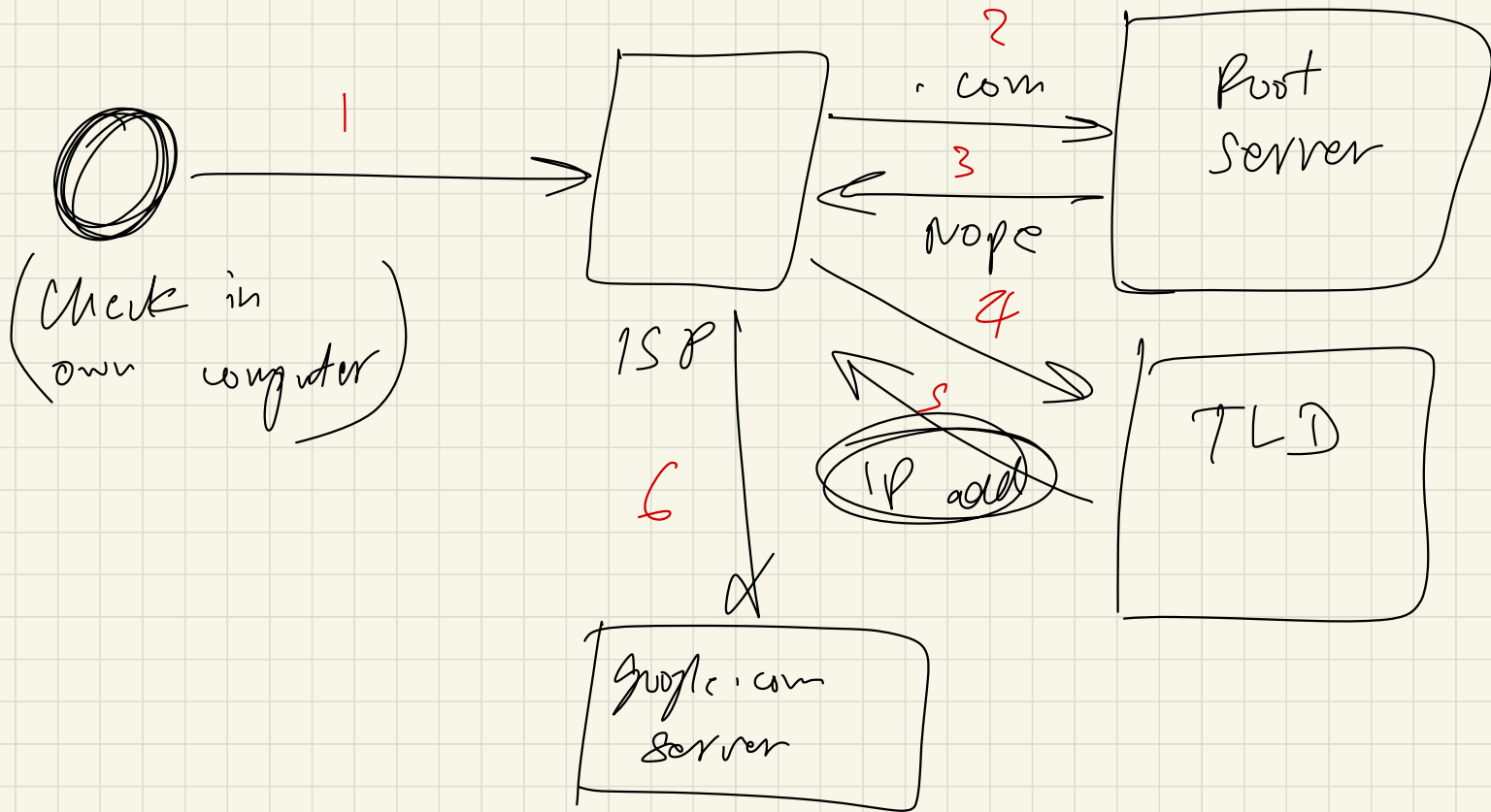
mail. google. com

Sub-domain Second-level domain Top level domain

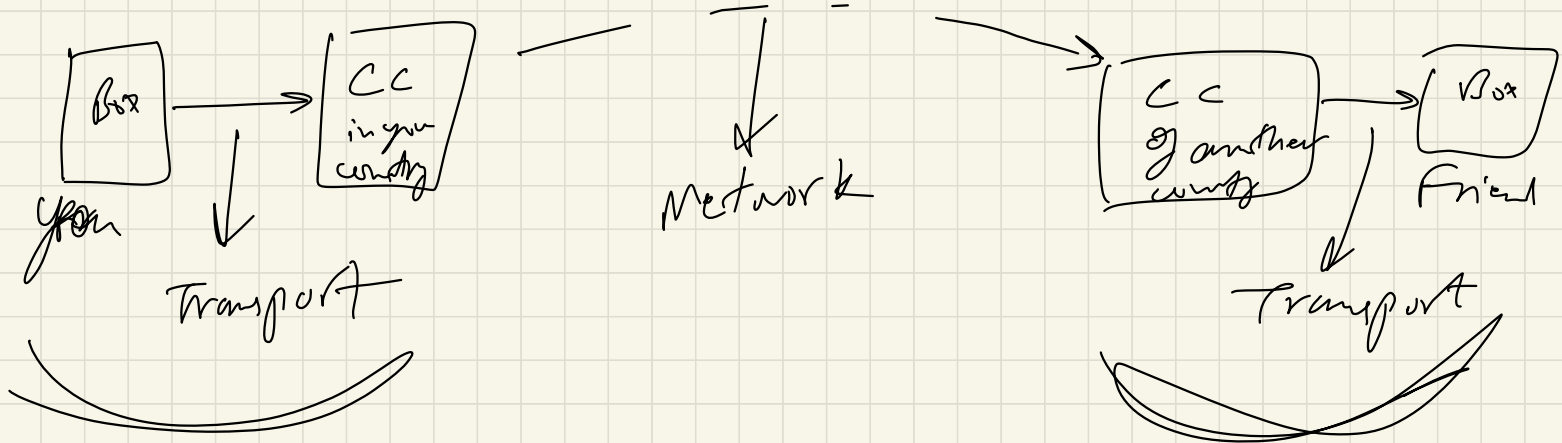
The diagram illustrates the hierarchy of the domain 'mail.google.com'. The word 'mail' is underlined with a bracket below it, and an arrow points from this bracket to the label 'Sub-domain'. The word 'google' is underlined with a bracket above it, and an arrow points from this bracket to the label 'Second-level domain'. The word 'com' is underlined with a bracket above it, and an arrow points from this bracket to the label 'Top level domain'. A horizontal line is drawn at the bottom right of the diagram.

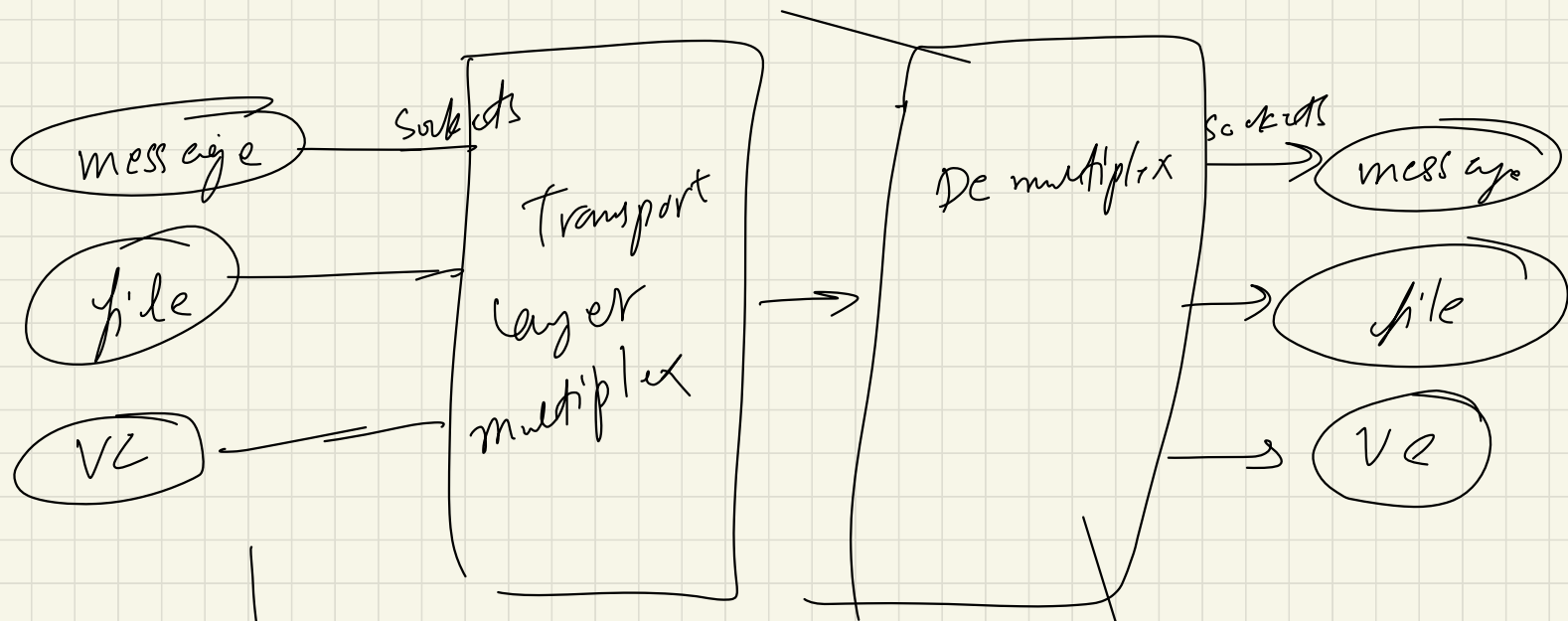
root DNS servers





② Transport Layer





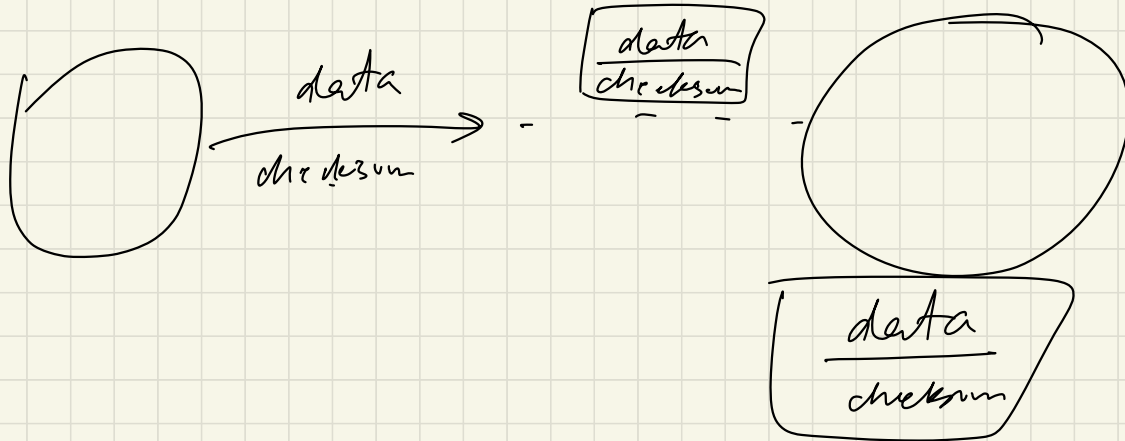
Data travels
in packets.

Transport layer
will attach these socket
port nos.

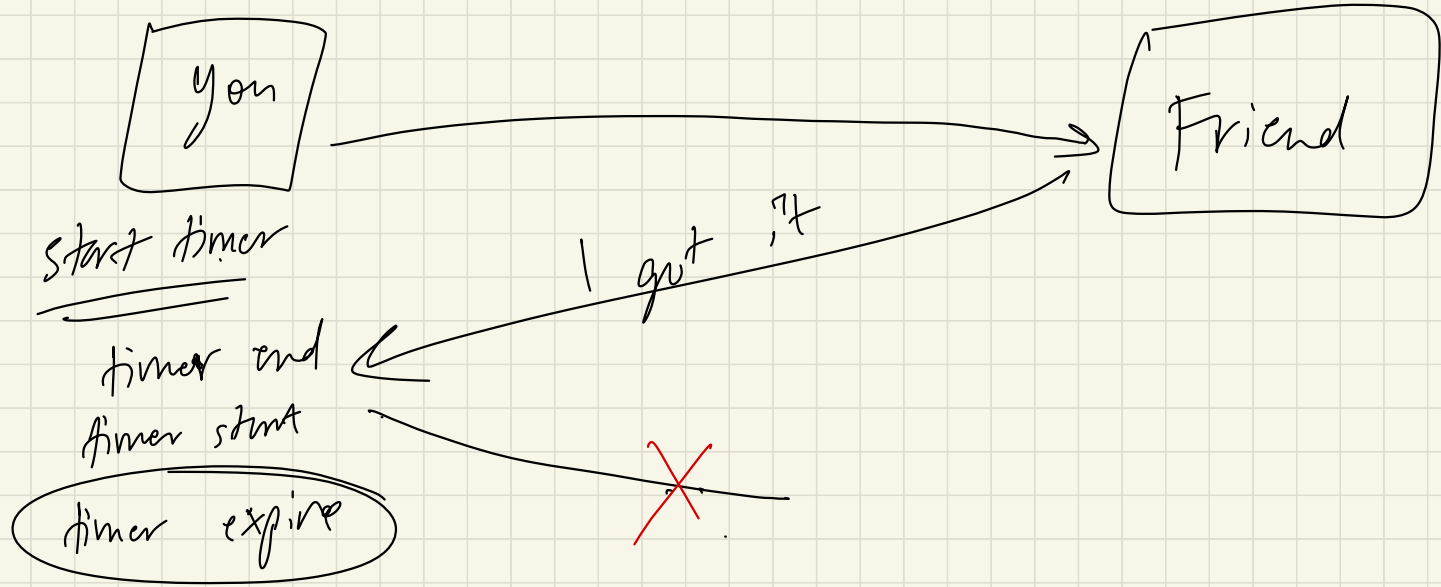
✦ Transport layer also takes care of congestion control.

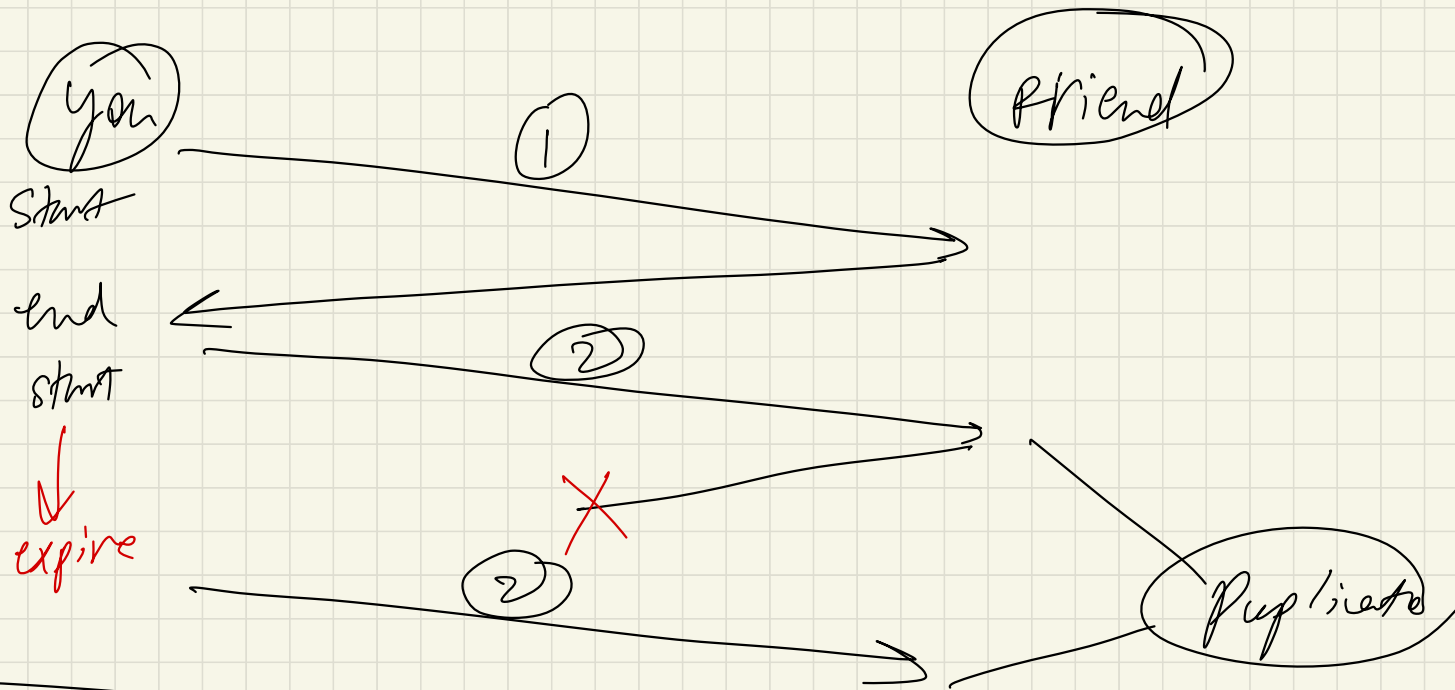
✦ Cong. control algorithms built in TCP.

Checksums:



Timers :





Solve this problem using
sequence nos.

UDP - User Datagram Protocol

- * Data may or may not be delivered.
- * Data may change
- * Data may not be in order

Connectionless

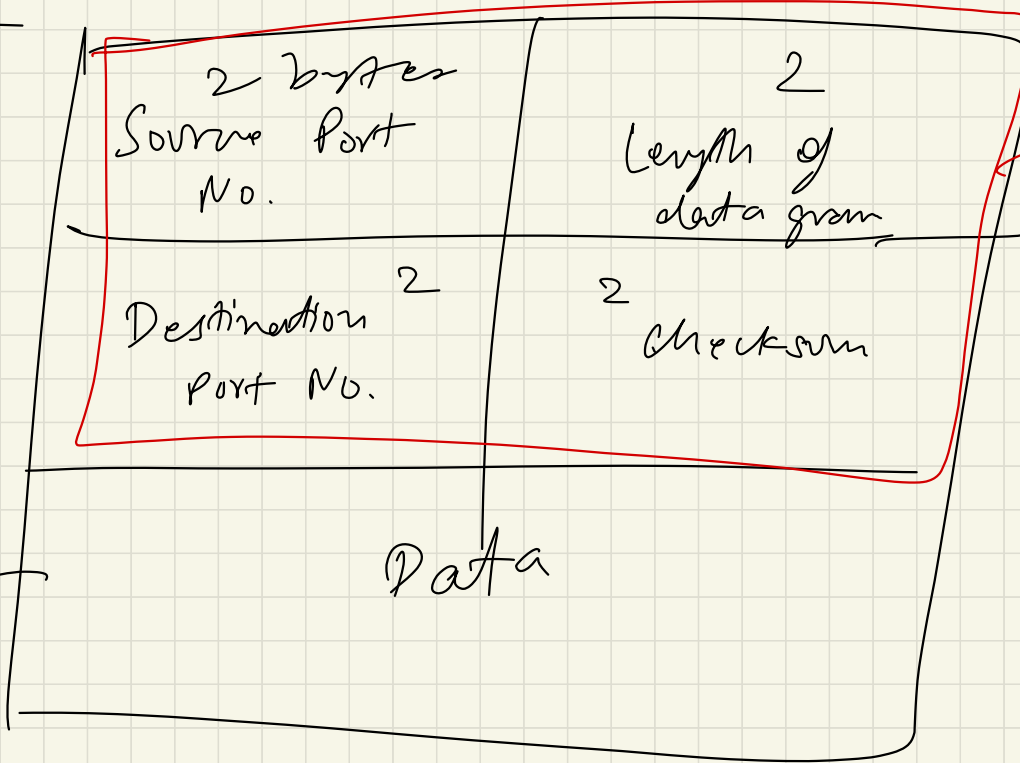
UDP uses checksums.

UDP Packet:

Total size
= 16

$$16 - 8$$

= 8 bytes



Header
& bytes

Uses Cases :

★ It's very fast

★ Video Conf appg

★ DNS → UDP

★ Gaming

TCP - Transmission Control Protocol

* Transport layer Protocol

* Application layer sends lots of raw data.

TCP segments this data → divide in chunks, add headers. It may also correct the data network layer → check video

* Congestion Control

* Takes care of :

— when data does not arrive

- maintains the order of data.

Features:

★ Connection oriented

★ Error control

★ Congestion control

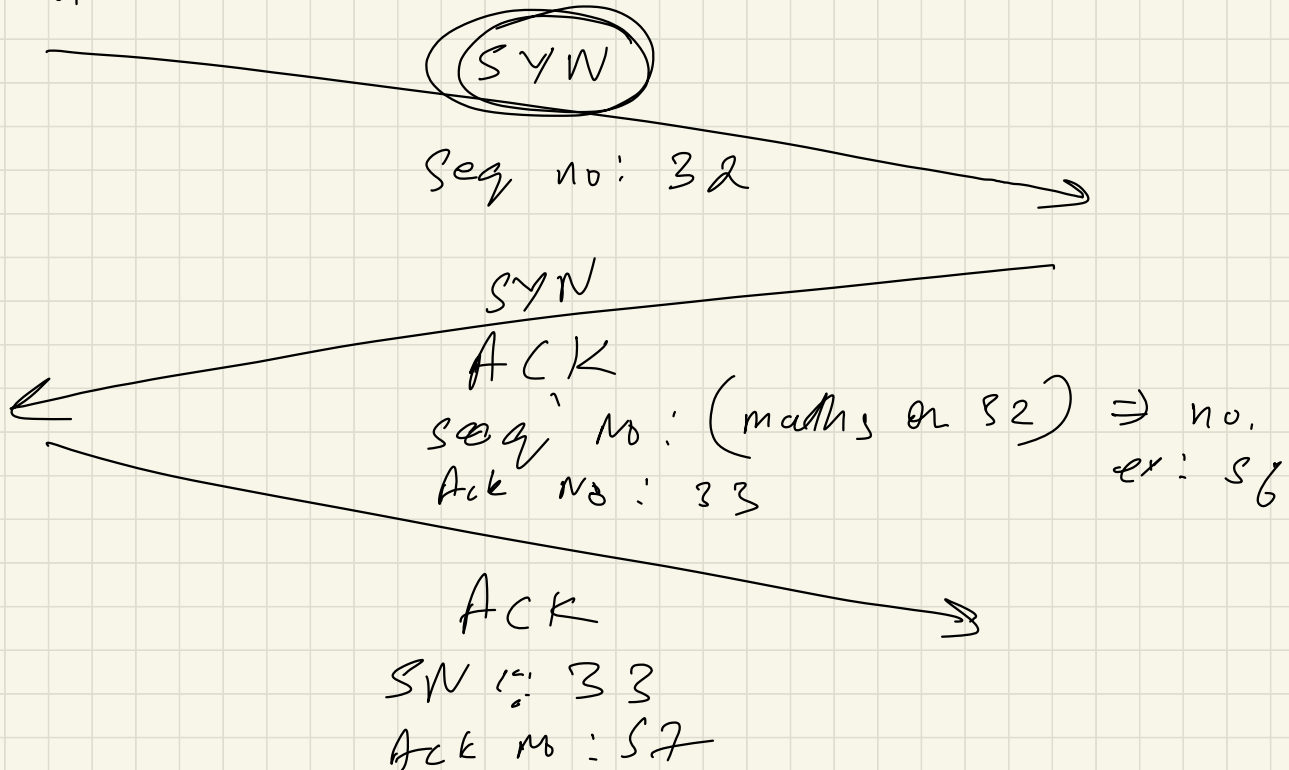
★ Full Duplex



3-way Handshake

Client

Server



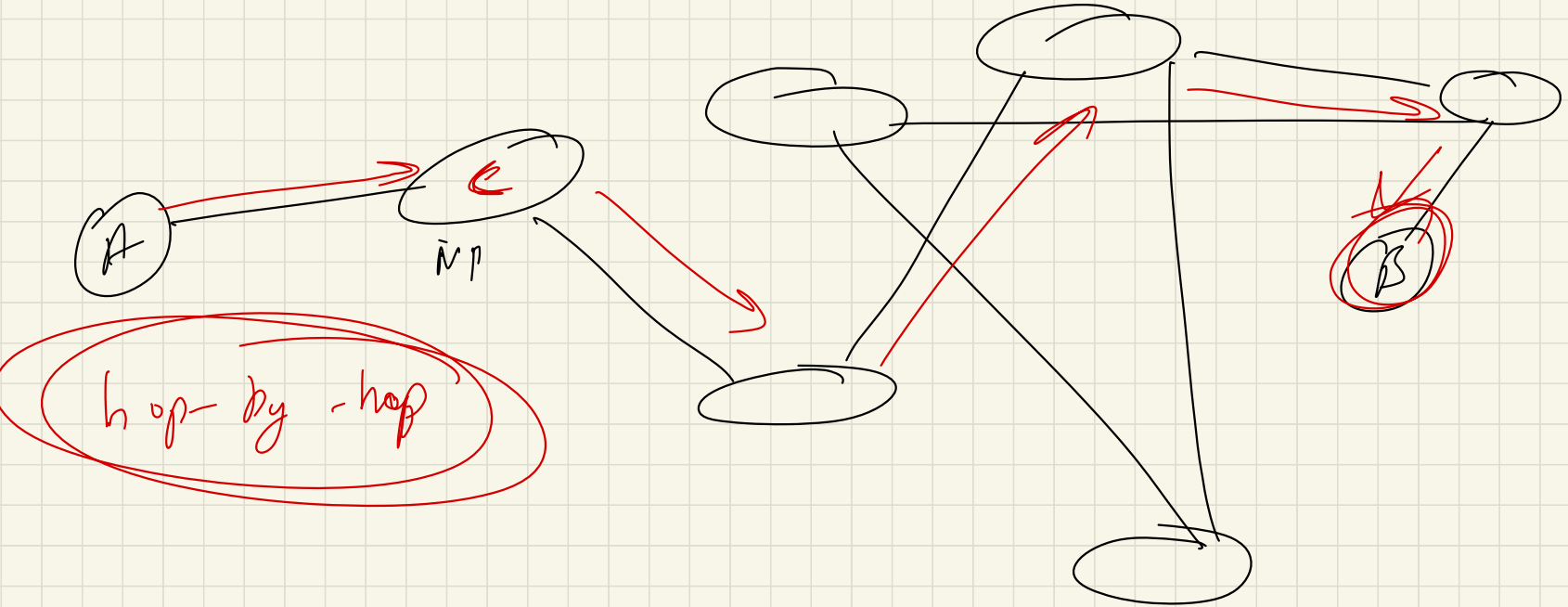
Network Layer

Here we work with routers.

✓ Transport → segments

Network → packets

Data link → frames



192.168.2.30

network address
Subnet ID

device
address
Host ID

Control Plane:

Routers → Nodes

Links → Edges

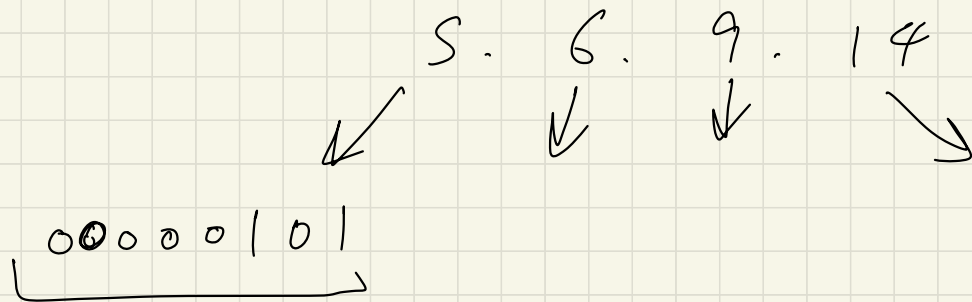
(i) Static Routing
& Manually

② Dynamic Routing

Internet Protocol (IP):

IPv4 $\rightarrow$ 32 bit, 4-words

IPv6 $\rightarrow$ 128-bits



Class of IP addresses:

✓✓ A

0.0.0.0

—

127.255.255.255

B

128.0.0.0

—

191.255.255.255

C

192.0.0.0

—

223.255.255.255

D

224.0.0.0

—

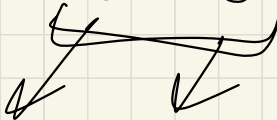
239.255.255.255

E

240.0.0.0

—

255.255.255.255

255. 255. 0. 0


128.0.0.0/31

192.0.1.0/24

start

end

192.0.1.0 192.0.1.255

256

Reserved addresses:

127. 0.0.0/8

Ex: localhost : 127.0.0.1

loopback addresses

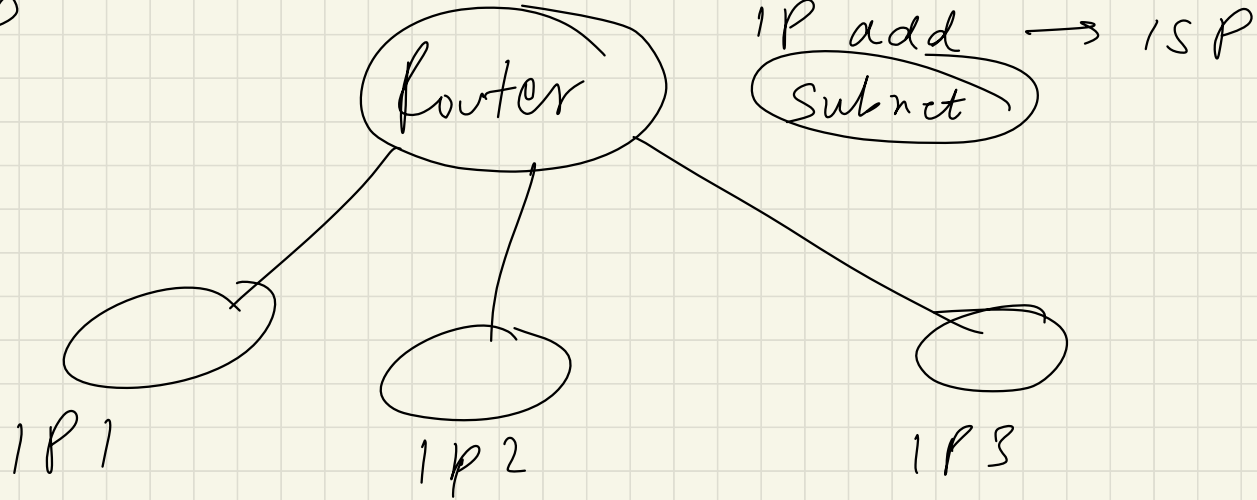
Packets: Header is of 20 bytes.

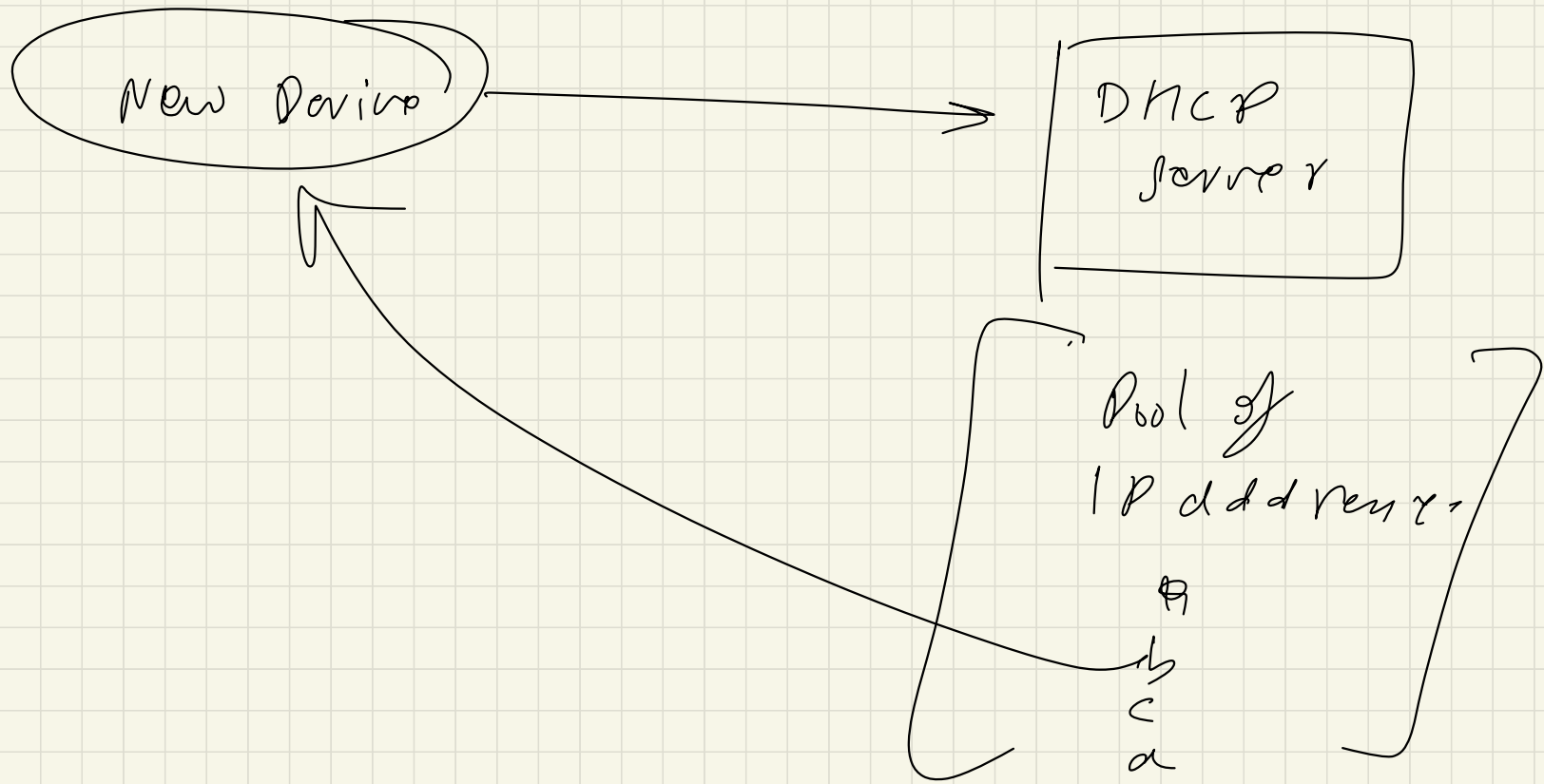
IPv, length, identification, flags, protocols,
checksum, address, TTL, etc.

Google

Data Link Layer

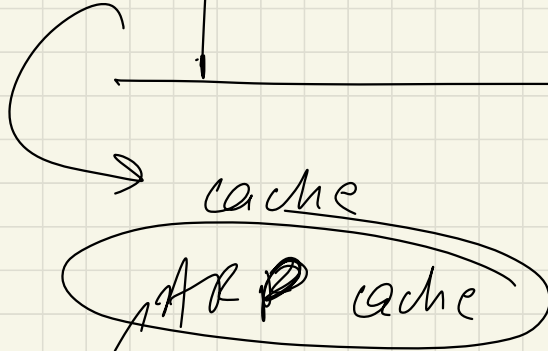
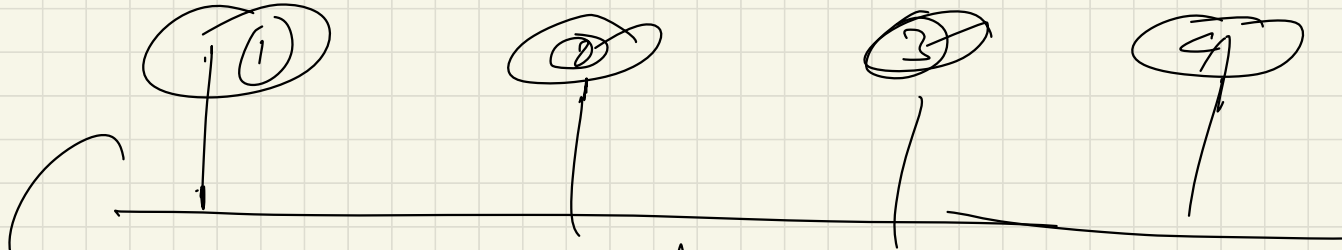
DHCP





many devices
connected in LAN :

data link layer add
MAC addresses



Address resolution
protocol

frame

DLLA of sender
& IP add of destination

★ framing

★ Error Detection_

IPv6

IPv4: $2^{32} \approx 4.3 \text{ billion}$

IPv6: $2^{32 \times 4} = 2^{128} = 3.4 \times 10^{38}$

Cons: $\nrightarrow$ Not Backward Compatible

$\nrightarrow$ ISPs would have to shift, lot of hardware work.

a : a : a : a : a : a : a : a

↓
~~Hexadecimal~~
16 bit

Ex:

ABFE : f001 : 3210 : 9182 : 0 : 0 : 1 : 3

127. 0. 0. 0 / 8 → prefix

→ we can do the same representation

ABFE : F001 : 0 : 9182 :: / 60

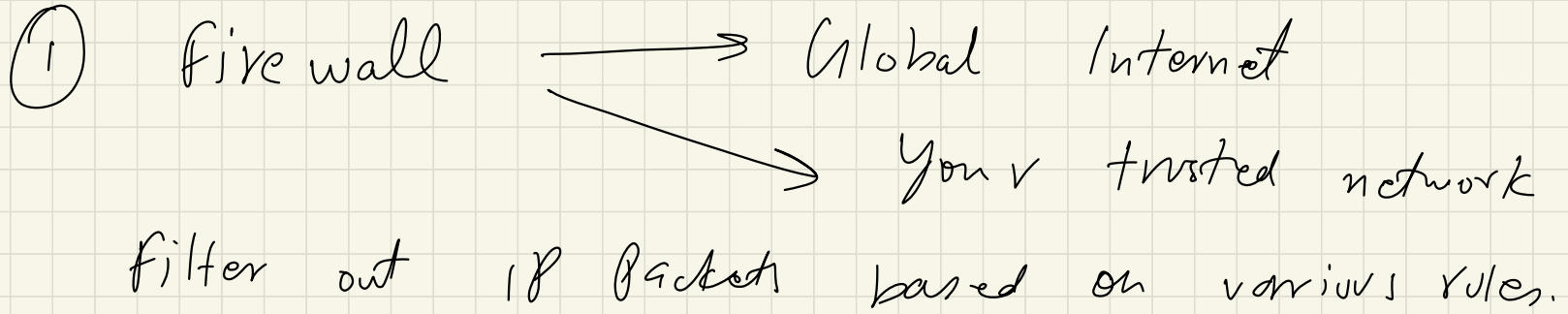
0000 $\longrightarrow$ 0

1 : 0000 : 0000 : 0000 : 9 = 1 : 0 : 0 : 0 : 9

= 1 : : 9

↓
the 0's

middleboxes:



→ Address

→ multiply packets

→ Port nos

→ flags

→ protocols

Stateless vs Stateful firewalls
→ more efficient

Name this in network . find system q/10.

② NAT : Network Address Translation

Watch video

